MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name:	Jensen Jacob Mathew	**Course:**	PCCST501 – Computer Networks
Roll No:	39	**Module:**	1
Class:	CSA	**Website Chosen:**	Amazon (www.amazon.com)

Introduction: When a user types `www.amazon.com` into a browser's address bar and presses Enter, a highly coordinated sequence of events unfolds across several layers of the Internet's protocol stack. This process spans from resolving human-readable domain names into machine-routable IP addresses to establishing secure channels and rendering the final graphical user interface. This report provides a detailed, step-by-step technical breakdown of this entire workflow.

1. The TCP/IP Protocol Stack and the Role of Each Layer

Communication over the Internet is structured using the TCP/IP model, which logically abstracts the complex process of moving data across heterogeneous networks into four distinct layers. Each layer relies on the services of the layer immediately below it and provides capabilities to the layer above it.

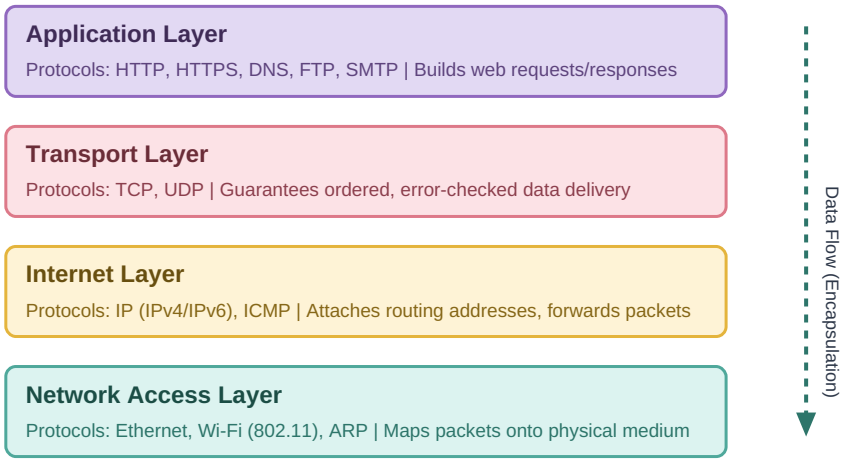


Figure 1: The four layers of the TCP/IP protocol stack and encapsulation direction.

- **Application Layer:** This is the highest level where user-facing applications (like Google Chrome or Safari) interface with network services. It generates data and formats the request to retrieve the Amazon page using application-layer protocols such as HTTP/HTTPS, and uses DNS to resolve domain names.

- **Transport Layer:** Responsible for host-to-host communication services. It establishes a logical link between the client and server. It takes the large application payload and chunks it into smaller blocks called segments. It uses **TCP (Transmission Control Protocol)** to ensure segments arrive reliably, in-order, and error-free via sequence numbers and acknowledgments.
- **Internet Layer:** Handles the addressing, packaging, and routing of packets across interconnected networks. It encapsulates the transport segments into IP packets by appending source and destination IP addresses. The primary protocol here is the **Internet Protocol (IP)**, which routers inspect to determine the next hop towards Amazon's data center.
- **Network Access (Link) Layer:** The lowest layer, responsible for transmitting raw bitstreams over a specific hardware medium (copper cables, fiber optics, or Wi-Fi radio waves). It wraps IP packets into network frames (e.g., Ethernet frames) and uses physical Media Access Control (MAC) addresses to guide frames locally.

2. The Role of DNS in Domain Resolution

Computers communicate across networks using numerical IP addresses, but humans use memorable strings like `www.amazon.com`. The **Domain Name System (DNS)** acts as the Internet's phonebook, converting human-readable domain names into machine-routable IP addresses.

When the user presses Enter, the browser first executes a lookup sequence to find the corresponding IP address:

1. **Local Cache Inspection:** The browser checks its internal DNS cache. If not found, it queries the Operating System's host cache.
2. **Recursive Resolver Query:** If the mapping isn't cached locally, a query is sent to the internet service provider's (ISP) or a public **DNS Recursive Resolver** (e.g., 8.8.8.8).
3. **Root Name Server Lookups:** If the resolver does not have the address cached, it queries a **Root Name Server** (`.`). The root server directs the resolver to the **Top-Level Domain (TLD) Server** responsible for `.com` extensions.
4. **TLD & Authoritative Server Queries:** The resolver queries the `.com` TLD server, which responds with the location of Amazon's **Authoritative Name Server**. Finally, the resolver queries the Authoritative server, which returns the actual IP address (e.g., `205.251.242.103`) along with a Time-To-Live (TTL) record.

The resolver returns this IP address back to the OS and browser, which caches it for future requests.

3. How HTTP/HTTPS Requests and Receives Web Pages

With the IP address resolved, the browser prepares to communicate with the web server using the **Hypertext Transfer Protocol (HTTP)** or its secure version, **HTTPS** (which runs over TLS/SSL encryption).

The transaction follows a strict Request-Response pattern:

- **The HTTP Request:** The browser generates a text-based request message. The first line contains the HTTP method (typically `GET` to fetch data), the resource path (`/` for the home page), and the protocol version (e.g., `HTTP/1.1` or `HTTP/2`). It follows up with headers such as `Host: www.amazon.com`, `User-Agent` (identifying browser details), and `Accept` rules.
- **The HTTP Response:** Upon processing the request, Amazon's web server replies with an HTTP response. The message starts with a status code indicating the outcome (e.g., `200 OK` for success, `301 Moved Permanently` for redirects, or `404 Not Found`). The response headers provide content length, server configuration, and cookie parameters, followed by a blank line and the actual payload body containing the page's raw HTML, CSS, and JavaScript elements.

4. How TCP Provides Reliable Communication for HTTP

Because HTTP is a stateless, high-level application protocol, it relies entirely on the Transport Layer's **TCP** to guarantee that web elements arrive perfectly intact without missing bytes or corrupted text.

TCP achieves absolute reliability through several key mechanisms:

- **The Three-Way Handshake:** Before any HTTP payload can be transmitted, TCP sets up a stable connection through a three-step greeting sequence:
  1. The client browser sends a `SYN` (Synchronize) packet containing an initial sequence number (x).
  2. Amazon's server responds with a `SYN-ACK` packet, acknowledging the client's number ($x + 1$) and sending its own initial sequence number (y).
  3. The client sends an `ACK` packet back ($y + 1$), establishing a full-duplex connection channel.
- **Sequence Numbers and Acknowledgments:** Every data segment is assigned a unique sequence number. When Amazon sends page elements, the browser must send back an acknowledgment (ACK) packet indicating it received those specific bytes. If a segment is dropped due to network congestion, the sender notices the missing ACK and retransmits the lost data.
- **Flow Control & Congestion Control:** TCP implements a sliding window mechanism. The browser informs the server how much buffer space it has left, preventing the server from overwhelming a slower client device. Congestion algorithms also dynamically scale down data speeds if network nodes start dropping packets.

5. Application Architecture: Client-Server Justification

The application architecture governing the Amazon web browsing experience is explicitly a **Client-Server Architecture**, rather than a Peer-to-Peer (P2P) network.

This structural classification is justified by the distinct, asymmetrical roles filled by the participants:

- **Asymmetric Roles:** The browser acts as a dedicated *Client*—a consumer that remains passive until a user prompts it to initiate a request. Amazon acts as the centralized, high-performance *Server*

infrastructure that listens continuously on well-known network ports (80 for HTTP, 443 for HTTPS) to fulfill incoming requests.

- **Centralization and Governance:** All core resources—including catalog listings, price databases, user authentication systems, and media storage—are held securely on Amazon's private, managed servers. Clients do not share data pieces with other shoppers browsing the website simultaneously. Instead, every client establishes an isolated, independent connection back to the central server cluster.

6. Architectural Comparison: HTTP vs. FTP

Feature / Metric	HTTP (Hypertext Transfer Protocol)	FTP (File Transfer Protocol)
Primary Objective	Designed for transferring web pages, hypermedia documents (HTML, CSS), and REST APIs.	Optimized specifically for bulk file transfers, uploading, downloading, and remote file manipulation.
Connection Model	Uses a single TCP connection per request/response sequence (in basic modes), blending control and data.	Employs out-of-band architecture: a **Control Connection** (Port 21) and a separate **Data Connection** (Port 20).
State Persistence	Stateless by design; each request is handled independently unless tracked via application cookies.	Stateful; maintains an active user session state, tracking working directories and authentication status.
Data Representation	Optimized for rendering short, transactional, or multi-element assets rapidly across web page components.	Tailored for large, continuous, binary or ASCII file streams with built-in transfer mode choices.

7. How Cookies Enhance User Experience

Because HTTP is inherently stateless, the server forgets who a client is as soon as a single request-response cycle finishes. To bridge this gap, servers use **Cookies**—small strings of textual data generated by the server, saved locally by the browser, and sent back automatically with every subsequent request to that same website.

> **Real-World Example (Amazon Shopping Cart Persistence):** Imagine browsing Amazon for laptop accessories. You find a mouse and click "Add to Cart". Without cookies, when you click onto the next page to find a keyboard, the server treats you as an entirely new visitor and your cart drops to zero items. By using a session cookie containing a secure identifier like `SessionID=XYZ987`, the server instantly matches your browser to your server-side database cart, ensuring items persist seamlessly as you browse or log back in later.

8. End-to-End Communication Flow Diagram

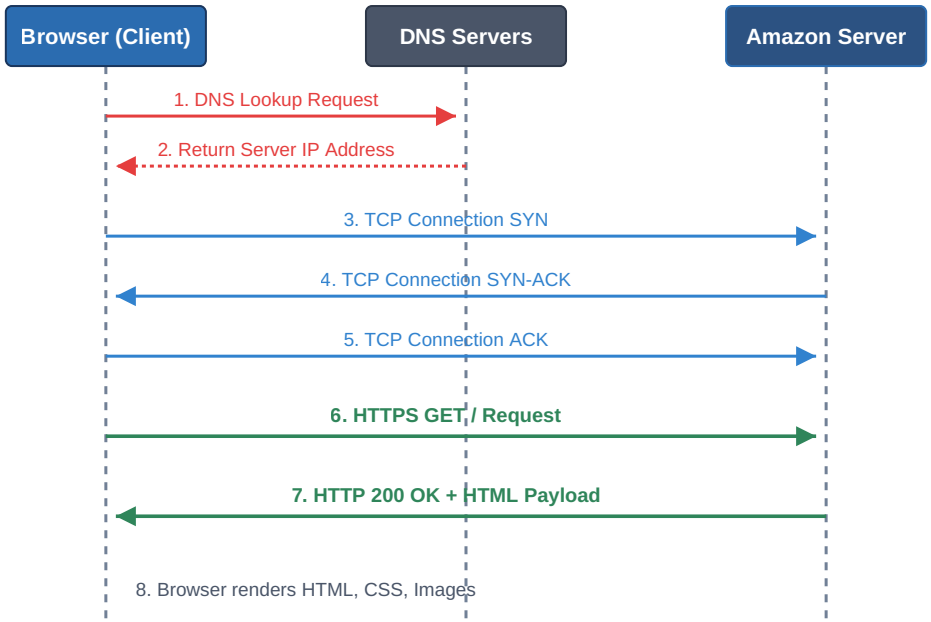


Figure 2: Sequence flow diagram of communication from client browser to server infrastructure.

9. Conclusion

Application-layer protocols represent the vital software interface that translates technical hardware capabilities into practical, daily internet activities. Without rules like HTTP/HTTPS, DNS, and SMTP, the Internet would be a restricted utility accessible only to network administrators via cryptic terminal scripts. These protocols define exactly how applications package, authenticate, and display diverse content variations across entirely different client devices. They isolate end users from the underlying complexities of routing tables, bit synchronization, and raw signal packaging. Ultimately, application-layer protocols act as the foundational bedrock of the modern digital economy, making complex networks completely transparent and turning global systems into single, predictable clicks.